ARCHITECTURESDEMO · .NET 10

Software architectures Cheat sheets

25 demos, one page each: the idea in one sentence, a diagram, the rules that make the architecture, when to use it, the pitfalls.

The same business everywhere: a small bank where Alice transfers €250 to Bob.

The details (code, tests, walkthroughs) are in the repository's `Docs/`, one full page per architecture; the no-code version is in `Docs/pour-tous/`.

Cheat sheets: one page per architecture

Each demo summed up, to reread in two minutes: the idea, a diagram, the rules, when to use it, the pitfalls. Each cheat sheet links to its full page. Code identifiers (`ComptesService` …) stay in French.

🖨️ **Printable version**: architecture-cheat-sheets.pdf (A4, one cheat sheet per page, 27 pages), regenerated by `python Docs/fiches/pdf/generer-pdf.py --en` (Chrome or Edge, access to jsDelivr).

📖 Docs · In French for now: 📗 for everyone · ★ cross-cutting concepts · ⚖️ differences

① Code organization: what goes where, who depends on whom?

#	Cheat sheet	The core idea
0	Layered	Dependencies flow down to the database
1	Hexagonal	Ports and adapters; everything points to the business core
2	Onion	The same rule, with a core split into layers
3	Clean	One interactor per use case, a presenter on the way out
4	Functional Core	Decide purely, apply in the shell
5	Vertical Slice	One file per feature
13	Microkernel	A core, plug-ins loaded at runtime
20	Pipeline	Filters in parallel, bounded pipes

② System decomposition: which boundaries, which deployment?

#	Cheat sheet	The core idea
6	Package by Component	One public interface, the rest `internal`
7	Modular Monolith	Modules connected by contracts, a single deployment
8	Microservices	One process and one database per service
12	Serverless	Stateless functions, triggered by events
14	Orchestrated Saga	An orchestrator drives and compensates
15	Multi-tenant	One application, several isolated customers
16	BFF	One backend per client application
17	Cells	N complete copies, bounded blast radius
18	Actors	Private state, messages one at a time, no locks
19	Space-based	Data in memory, database updated later
21	Choreography	Each one reacts to the others' events

③ Modeling: how to represent the business and the state?

#	Cheat sheet	The core idea
9	CQRS	One model to write, another to read
10	Event Sourcing	Store the events, replay the state
11	DDD	The business language, bounded contexts

④ Migration · ⑤ Exposure · ⑥ Presentation

#	Cheat sheet	The core idea
22	Strangler Fig	A façade, one route migrated at a time, reversible
23	REST, gRPC, GraphQL	Resources, procedures, a graph chosen by the client
24	MVC and MVVM	A view discarded on every request, or a living ViewModel

Three reflexes common to all the cheat sheets

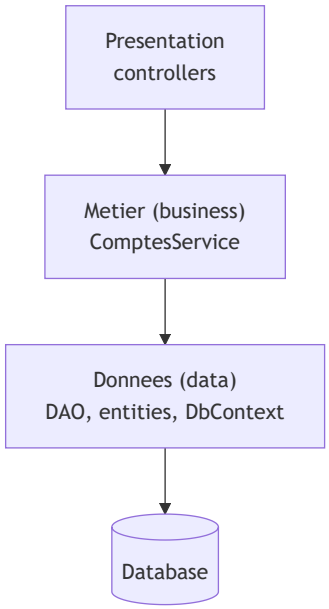
1. **Start simple** (Layered, Vertical Slice, Modular Monolith); add complexity only for a real problem.
2. **Enforce the architecture** with the compiler (`internal`, project references) or an architecture test, otherwise it gets bypassed.
3. **Network or messages?** Idempotency, outbox, eventual consistency, compensation (cross-cutting concepts).

Cheat sheet 0 · Layered (N-tier)

French name: *Architecture en couches (N-tiers)*

> **In one sentence**: Presentation → Business → Data; each layer only calls the one below it.

Family: code organization · **Origin**: common practice since the 1990s · **Demo**: `00-Couches` (port 5100) · 📖 Full page



Key points

- Responsibilities are split along **technical** lines and stacked.
- **Dependencies flow down to the database**: the business layer depends on the data layer (its interface, its entities).
- Entities are **anemic**: the rules live in the services.
- Safeguard: **non-transitive** project references (the presentation cannot see the data layer).

✅ When to use it	❌ Signs it's time to move on
CRUD with thin business logic, prototype, junior team	Rules multiply and need testing without a database
Starting fast, with a structure everyone knows	A change of storage is on the table

Pitfall: one feature = changes in every layer; the business logic is coupled to persistence.

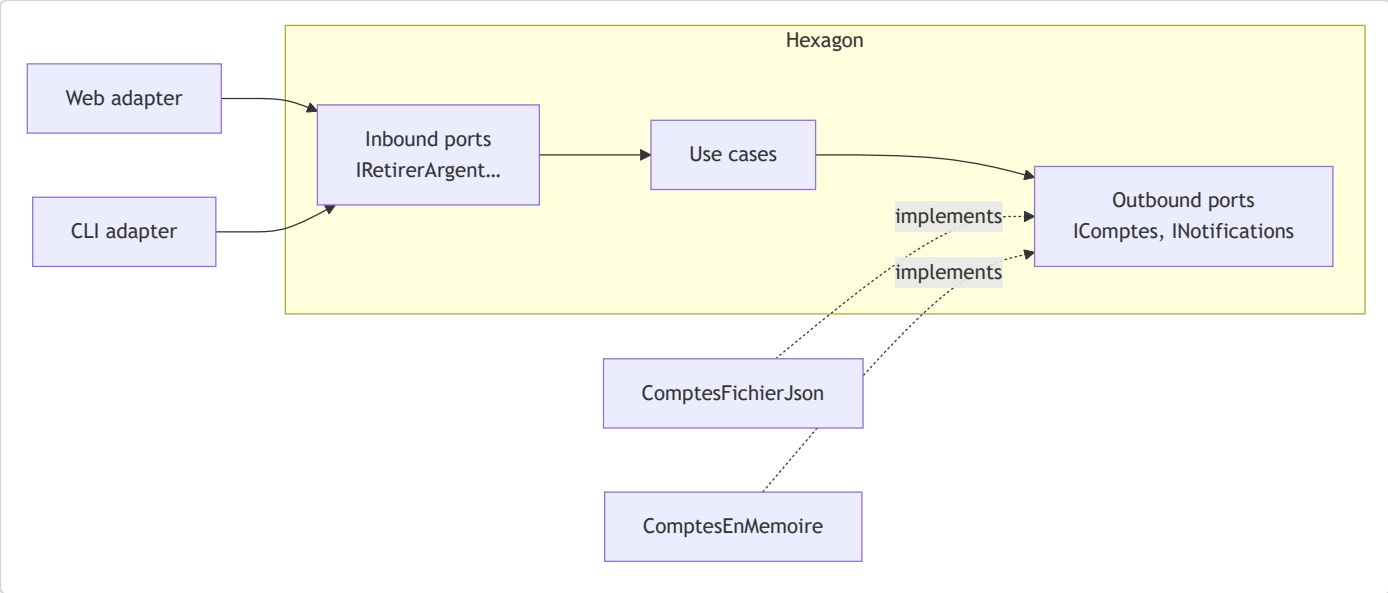
Next steps: invert the dependencies → Hexagonal, Onion; organize by feature → Vertical Slice.

Cheat sheet 1 · Hexagonal (Ports & Adapters)

French name: *Architecture hexagonale*

> **In one sentence**: the business core depends on nothing; the outside world plugs into it through **ports** (interfaces) and interchangeable **adapters** (implementations).

Family: code organization · **Origin**: Alistair Cockburn, 2005 · **Demo**: `01-Hexagonale` (port 5101, + a CLI) · 📖 Full page



Key points

- Inside / outside rather than top / bottom. **Every dependency points toward the hexagon**.
- **Inbound** port = what the application offers; **outbound** port = what it needs.
- The *composition root* (`Program.cs`) is the only place that knows both the hexagon and the adapters.
- **Contract tests**: the same suite runs against every adapter of a port.

✅ When to use it	❌ Less useful if
Rich business logic, infrastructure likely to change	Simple CRUD with no rules
Several input channels (API, CLI, messages)	One adapter per port, forever

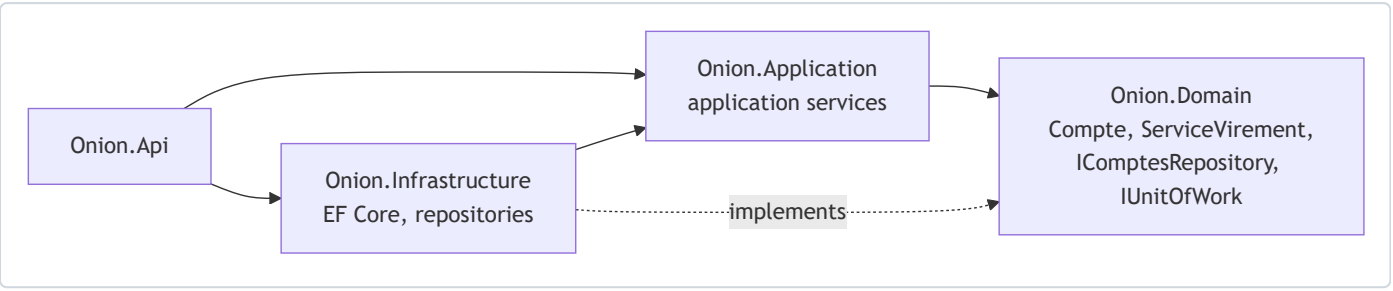
Pitfall: pass-through ports that add nothing; the core itself is left unstructured.

Neighbors: same dependency rule as Onion and Clean.

Cheat sheet 2 · Onion

> **In one sentence**: concentric layers; the domain at the center, **every dependency points toward it**.

Family: code organization · **Origin**: Jeffrey Palermo, 2008 · **Demo**: `02-Onion` (port 5102) · 📖 Full page



Key points

- Same rule as hexagonal, but the **center is split**: domain model → domain services → application services.
- The data-access interfaces (`IComptesRepository`, `IUnitOfWork`) live **in the domain**; the infrastructure implements them.
- The application service orchestrates: load, delegate to the domain, save (unit of work, all or nothing).

✅ When to use it	❌ Less useful if
Rich business logic, with rules spanning several entities	Small application: lots of projects and interfaces
Need for a structured core and an explicit unit of work	Team that doesn't distinguish domain from application services

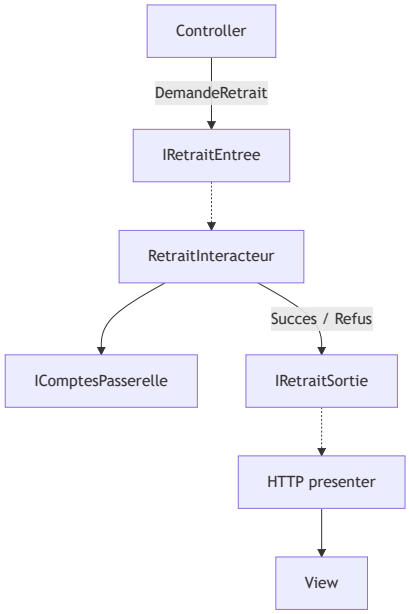
Pitfall: the rules drift into the services → anemic model.

Hexagonal or Onion? Same rule; one stresses the ports around the core, the other the layers inside it. Many ".NET Clean" projects are actually Onion.

Cheat sheet 3 · Clean Architecture

> **In one sentence**: one *interactor* per use case, explicit boundaries; it returns nothing and pushes its result to a *presenter*.

Family: code organization · **Origin**: Robert C. Martin, 2012 · **Demo**: `03-Clean` (port 5103) · 📖 Full page



Key points

- Four circles: **Entities** → **Use Cases** → **Interface Adapters** → **Frameworks & Drivers**.
- **Dependency rule**: a circle only knows the circles further in.
- Vocabulary: request / response model, input / output boundary, interactor, presenter, gateway.
- Dependency inversion also applies to the **output**: the interactor calls the presenter through an interface it defines.

✅ When to use it	❌ Less useful if
Several presentations of the same use case	Simple web API: a stateful presenter feels unnatural in ASP.NET Core
Many use cases worth making explicit	Team that would apply it dogmatically

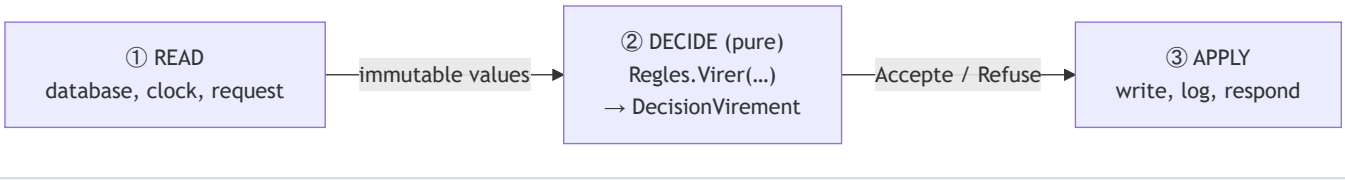
Pitfall: the **ceremony** (6 types for one withdrawal). In practice, the .NET templates (Jason Taylor, Ardalis) keep the circles and replace the presenter with a `Result<T>`.

Compare: the same withdrawal fits in one file with Vertical Slice.

Cheat sheet 4 · Functional Core, Imperative Shell

> **In one sentence**: **pure** rules that return a decision; a thin **shell** around them does all the input/output.

Family: code organization · **Origin**: Gary Bernhardt, "Boundaries", 2012 · **Demo**: `04-CoeurFonctionnel` (port 5104) · 📖 Full page



Key points

- The core: no `async`, no interfaces, no database, no clock. **Values in, a decision out.**
- The shell: imperative, thin, **makes no decisions**.
- Decisions are **data**: they can be logged and replayed.
- Core tests: values → value, **without a single mock**, in milliseconds.

✅ When to use it	❌ Hard if
Complex rules: pricing, eligibility, limits, calculations	The decision requires cascading reads
Inside any other architecture	The team isn't used to the functional style

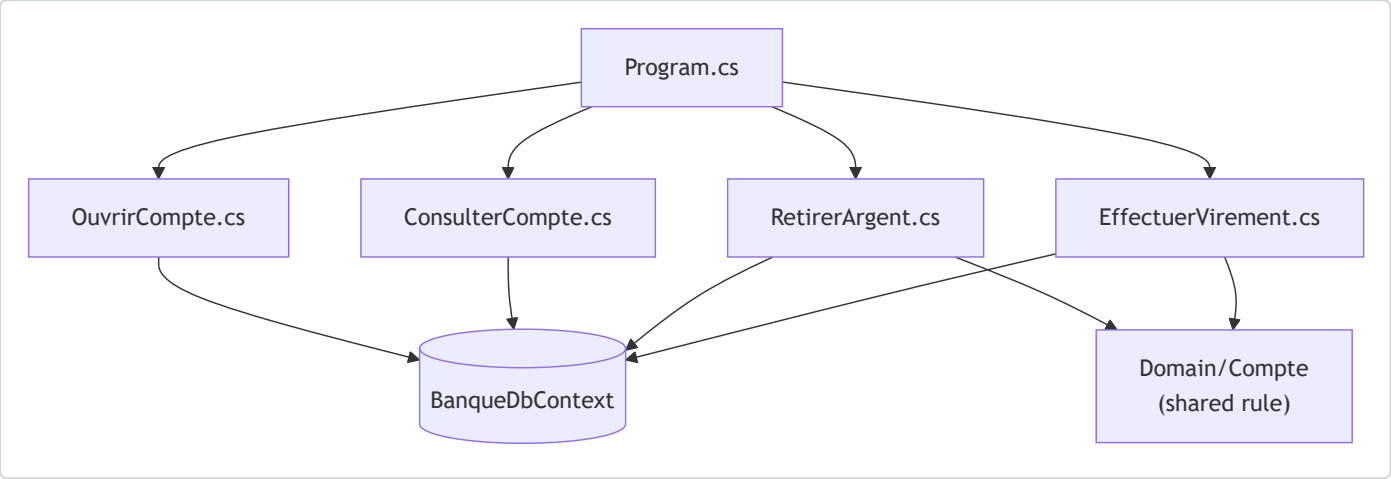
Pitfall: you have to **load everything before** deciding.

Neighbors: hexagonal taken to the extreme; the same "decide / apply" split is at the heart of Event Sourcing.

Cheat sheet 5 · Vertical Slice

> **In one sentence**: code organized **by feature**; each slice contains everything, from the HTTP route down to the database.

Family: code organization · **Origin**: Jimmy Bogard, around 2018 · **Demo**: `05-VerticalSlice` (port 5105) · 📖 Full page



Key points

- One slice = one query or one command, in **one file**.
- **Tight coupling within** a slice, **loose coupling between** slices.
- Each slice picks its own technique (LINQ projection, entity, SQL…).
- No "just in case" abstractions: the `DbContext` already is a repository + unit of work. Only **shared** rules move up into `Domain/`.

✅ When to use it	❌ Less suited if
Line-of-business APIs, delivered feature by feature	A database change is likely (the `DbContext` is everywhere)
Inside each module of a modular monolith	Team with little discipline about shared code

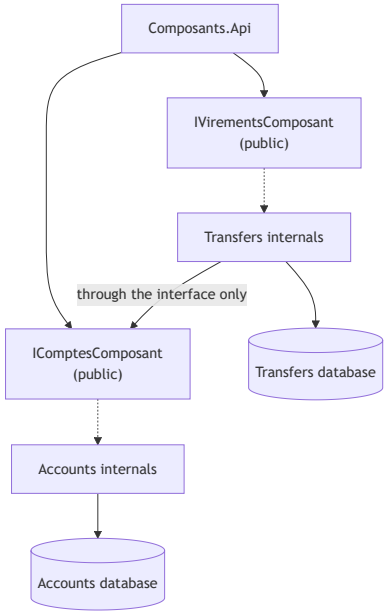
Pitfall: duplication and bloated handlers if nothing is ever moved up. (MediatR has required a paid license for companies since 2025.)

Neighbors: the vertical axis continues with Package by Component then Modular Monolith.

Cheat sheet 6 · Package by Component

> **In one sentence**: a **single public interface** per component, everything else `internal`; the compiler enforces the architecture.

Family: system decomposition · **Origin**: Simon Brown, "The Missing Chapter" (*Clean Architecture*) · **Demo**: `06-Composants` (port 5106) · 📖 Full page



Key points

- Observation: if everything is `public`, nothing stops a controller from calling the repository directly.
- A component = business logic + data behind an interface; a shortcut **does not compile**.
- **One database per component**: Transfers never reads the Accounts tables.

✅ When to use it	❌ Cost
Enforcing boundaries without tests or reviews	Designing good public interfaces takes care
Preparing a future modular monolith	No more shared transaction: compensation is needed

Takeaway: an architecture protected by neither `internal` nor an architecture test (NetArchTest, ArchUnitNET) always ends up bypassed.

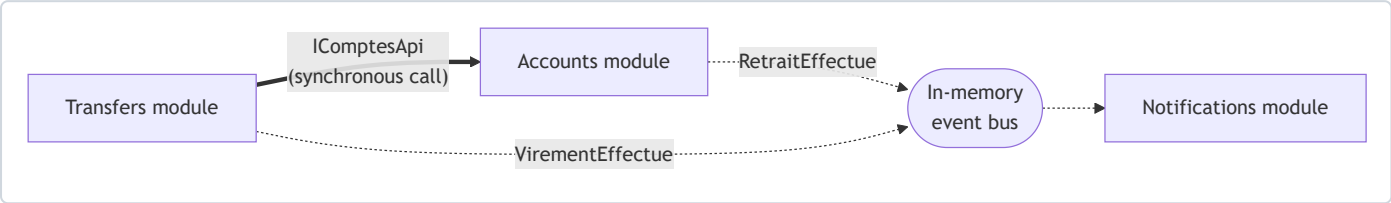
Next step: components that also own their API and their events → Modular Monolith.

Cheat sheet 7 · Modular Monolith

French name: *Monolithe modulaire*

> **In one sentence**: autonomous modules (API, database, events) connected by **contracts**, deployed together in a single process.

Family: system decomposition · **Origin**: popularized notably by Shopify · **Demo**: `07-MonolitheModulaire` (port 5107) · 📖 Full page



Key points

- A module = a mini-application (routes, database, subscriptions) plugged in with one line: `IModule[] modules = [...]`.
- Rules, checked by architecture tests:
  1. you only reference another module's **Contracts**, never its implementation;
  2. only the `IModule` class is public;
  3. contracts depend on no module;
  4. **one database per module**.

✅ When to use it	❌ Limits
Recommended starting point for a growing system	Modules can't be scaled independently
Several subdomains, a single operations team	No transactions or joins across modules

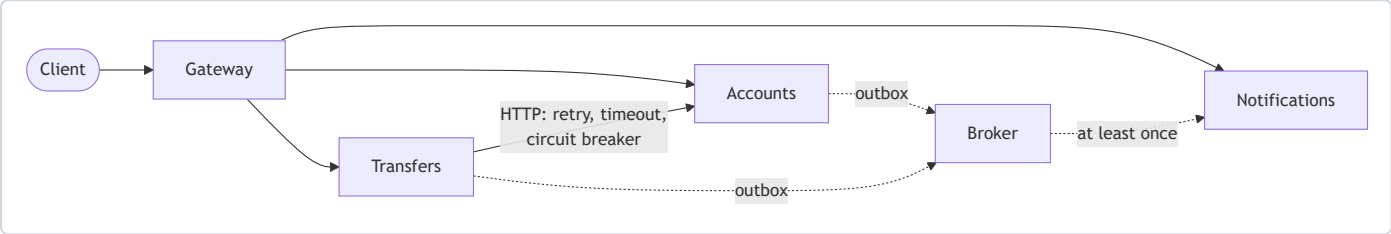
Pitfall: the in-memory bus loses events on a crash → outbox. A bad split is expensive: find the boundaries with DDD.

Next step: a module that must be deployed separately → its contracts become a network API → Microservices.

Cheat sheet 8 · Microservices

> **In one sentence**: **one process per service**, one database each, deployable on its own; communication over the **network** (HTTP + messages), no shared code.

Family: system decomposition · **Demo**: `08-Microservices` (gateway 5200, services 5201–5203, broker 5209; start everything with `Micro.Lanceur`) · 📖 Full page



Key points

- Demo 7 cut into processes: method calls, transactions and in-memory events now go through **a network that can fail**.
- The contract is the **exchanged JSON**: no references between services.
- Mandatory mechanisms: **idempotency** (`operationId`, `Message-Id`), **outbox**, resilient client, **saga + compensation**, a gateway that hides the `/interne/…` routes.

✅ When to use it	❌ Avoid if
Several teams, independent release cycles	A single team: the modular monolith is enough
Very different scaling needs	No operations tooling (tracing, monitoring)

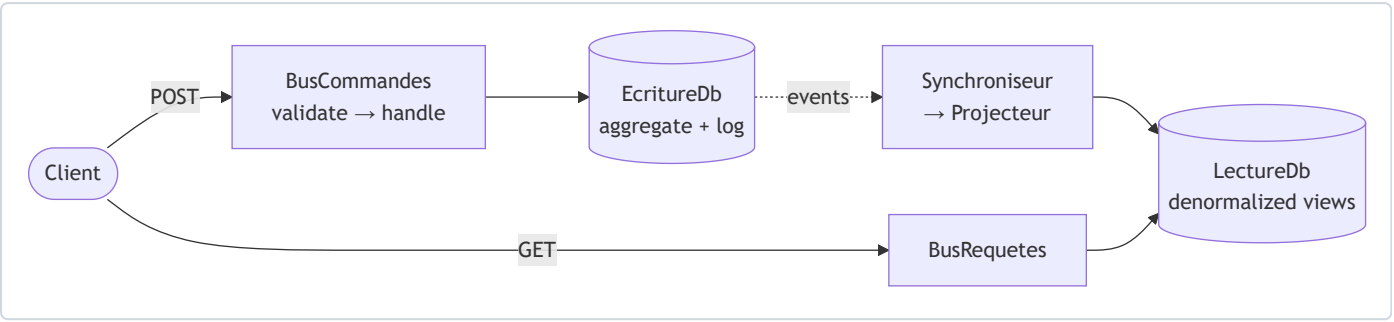
Pitfall: partial failures and eventual consistency everywhere; most of the code handles failures.

In production: RabbitMQ / Azure Service Bus, YARP, .NET Aspire or Kubernetes, OpenTelemetry. Mechanisms: cross-cutting concepts.

Cheat sheet 9 · CQRS

> **In one sentence**: one model to **write** (the rules), another to **read** (views ready to display), connected by events.

Family: modeling · **Origin**: Greg Young, around 2010, after Bertrand Meyer's CQS · **Demo**: `09-CQRS` (port 5108) · 📖 Full page



Key points

- Writing and reading have different needs: consistency and invariants on one side, speed and varied shapes on the other.
- A command returns nothing (or an identifier); a query changes nothing.
- The two sides **don't reference each other**: their only link is the events project.
- The views can be **rebuilt** from the events.

✅ When to use it	❌ Overkill if
Reads far more numerous than, or very different from, writes	Simple CRUD
Screens that want precomputed views	Immediate consistency is required everywhere

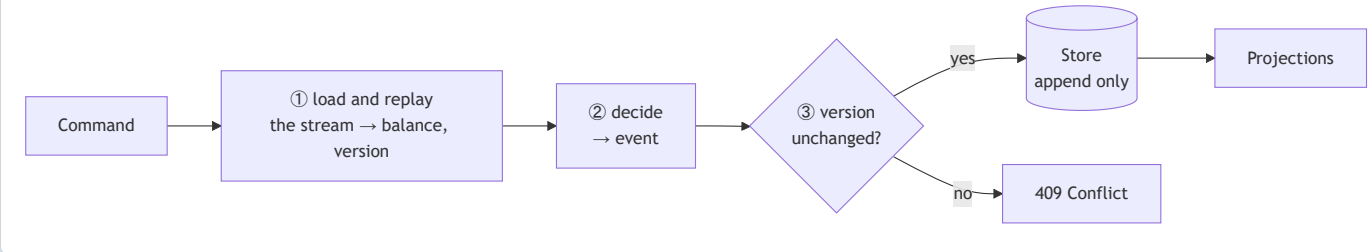
Vocabulary traps: CQRS **does not require two databases** (separating the models over one database is often enough); CQRS **is not** Event Sourcing (here, the state is stored normally).

Cost: duplicated data, projections to maintain, eventual consistency to explain.

Cheat sheet 10 · Event Sourcing

> **In one sentence**: you store the **sequence of events**, not the state; the state is recomputed by replaying them, and nothing is ever deleted.

Family: modeling · **Demo**: `10-EventSourcing` (port 5109; human-readable store: `Es.Api\evenements.jsonl`) · 📖 Full page



Key points

- The balance is only a **calculation** over the facts (opened, withdrawn, transferred…), as in accounting.
- Two-step aggregate: **decide** (rules → event) then **apply** (event → state).
- **Optimistic concurrency** by version number; **snapshots** to avoid replaying everything.
- **Upcasting** of old event versions; **Given / When / Then** tests.

✅ When to use it	❌ Overkill if
History is business data: banking, accounting, insurance, audit	CRUD, or a module with no need for history
Time travel, new retroactive views	Team in a hurry: the learning curve is real

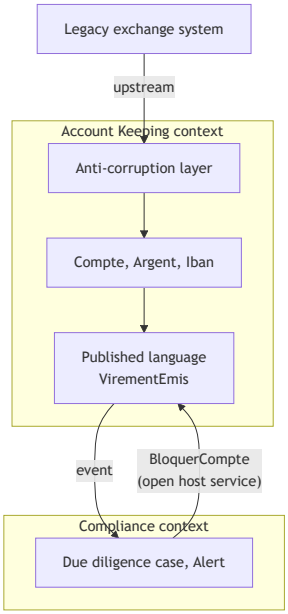
Pitfalls: you correct with a **compensating** event; events are **versioned forever**; **GDPR**: encrypt personal data and destroy the key.

Neighbors: almost always paired with CQRS for reads. In production: EventStoreDB, Marten, Axon.

Cheat sheet 11 · Domain-Driven Design (DDD)

> **In one sentence**: start from the business, speak **its** language, and let the model drive the code, both within a context (tactical) and between contexts (strategic).

Family: modeling · **Origin**: Eric Evans, *Domain-Driven Design*, 2003 · **Demo**: `11-DDD` (port 5110, Onion base) · 📖 Full page



Key points

- **Strategic**: bounded contexts (an "account" is not the same thing in Account Keeping and in Compliance), context map, published language, anti-corruption layer.
- **Tactical**: ubiquitous language (glossary `LANGAGE.md` = names in the code), **always-valid value objects** (`Iban`, `Argent`), aggregates, factories, domain services, repositories, domain events.
- It is **not a folder layout** but an approach, usable with Hexagonal, Onion or Clean.

✅ When to use it	❌ Overkill if
Complex business, experts available	CRUD: a lot of ceremony
Finding **where to cut** modules and services	No access to domain experts

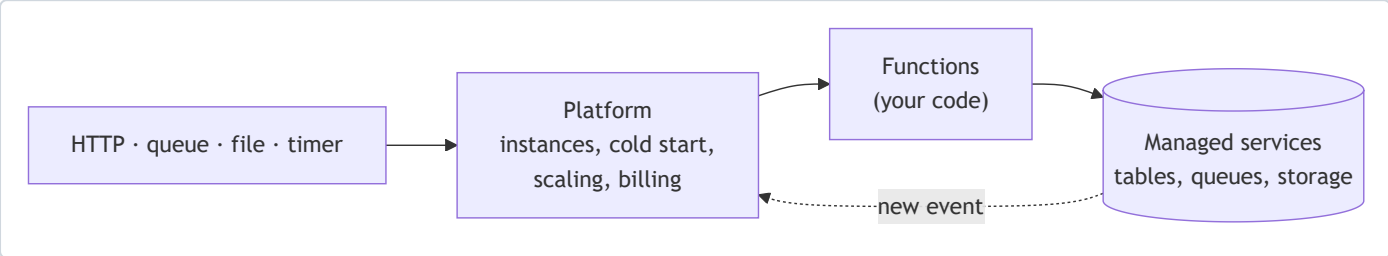
Pitfall: reducing it to its tactical toolbox; splitting contexts is hard and expensive to fix.

Links: contexts → modules (7) or services (8); aggregates → event streams (10).

Cheat sheet 12 · Serverless (FaaS)

> **In one sentence**: **stateless functions** run on demand when an **event** occurs, on servers managed by the provider, billed per use.

Family: system decomposition · **Origin**: AWS Lambda, 2014 · **Demo**: `12-Serverless` (port 5111, **simulated** platform) · 📖 Full page



Key points

- "Serverless" = **no servers to manage**. You write short functions and a **manifest** (which function, which trigger).
- **Event-driven** loop: a function writes to a queue or to storage, which triggers another one.
- State lives only in **managed services** (concurrency via ETag); queues with a *poison* queue.
- In the demo, `Sls.Plateforme` plays the provider; only `Sls.Fonctions` would be written for real.

✅ When to use it	❌ Avoid if
Files, messages, webhooks, scheduled jobs	Heavy constant load (high cost)
APIs with irregular traffic, "glue" between cloud services	Strict latency (cold starts), long-running work

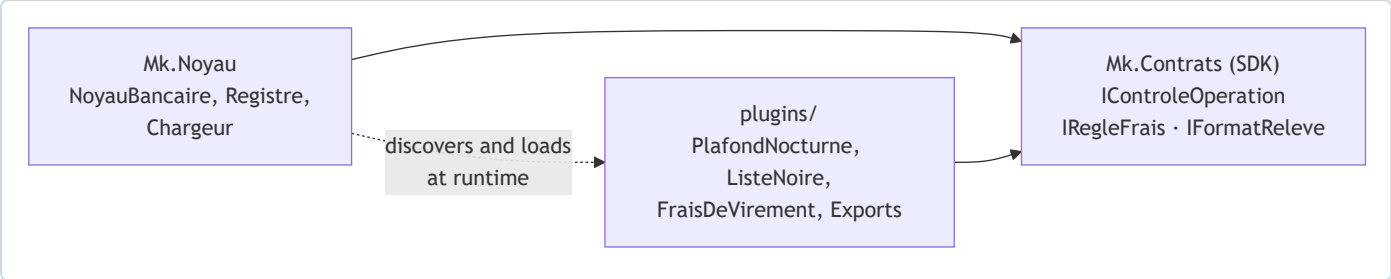
Pitfalls: **idempotency** is mandatory (multiple deliveries), provider lock-in, duration and memory limits.

Going real: Azure Functions (isolated worker) or AWS Lambda, Azurite / LocalStack, Durable Functions / Step Functions to orchestrate.

Cheat sheet 13 · Microkernel (plug-ins)

> **In one sentence**: a **minimal core** that defines extension points; the variable features are **plug-ins** loaded, unloaded and updated at runtime.

Family: code organization · **Examples**: VS Code, Eclipse, web browsers, payroll software · **Demo**: `13-Microkernel` (port 5112; build the solution first, `plugins/` is a build output) · 📖 Full page



Key points

- The core references **no** plug-in; plug-ins only see the **SDK**.
- Three extension points: **reject** an operation, **add fees**, **produce a statement** (one DLL can contain several).
- One load context per plug-in: unloading is possible, settings come from `plugin.json`, the folder is watched.
- A faulty plug-in is set aside.

✅ When to use it	❌ Be careful if
Product configurable per customer, country, offer	Unstable extension points: changing them breaks every plug-in
Rule engines, third-party extensions	Untrusted plug-ins: they run **in-process**

Pitfalls: copying the SDK into the plug-in folder (incompatible types); holding a reference to a plug-in (unloading becomes impossible); the SDK is a public contract to **version**.

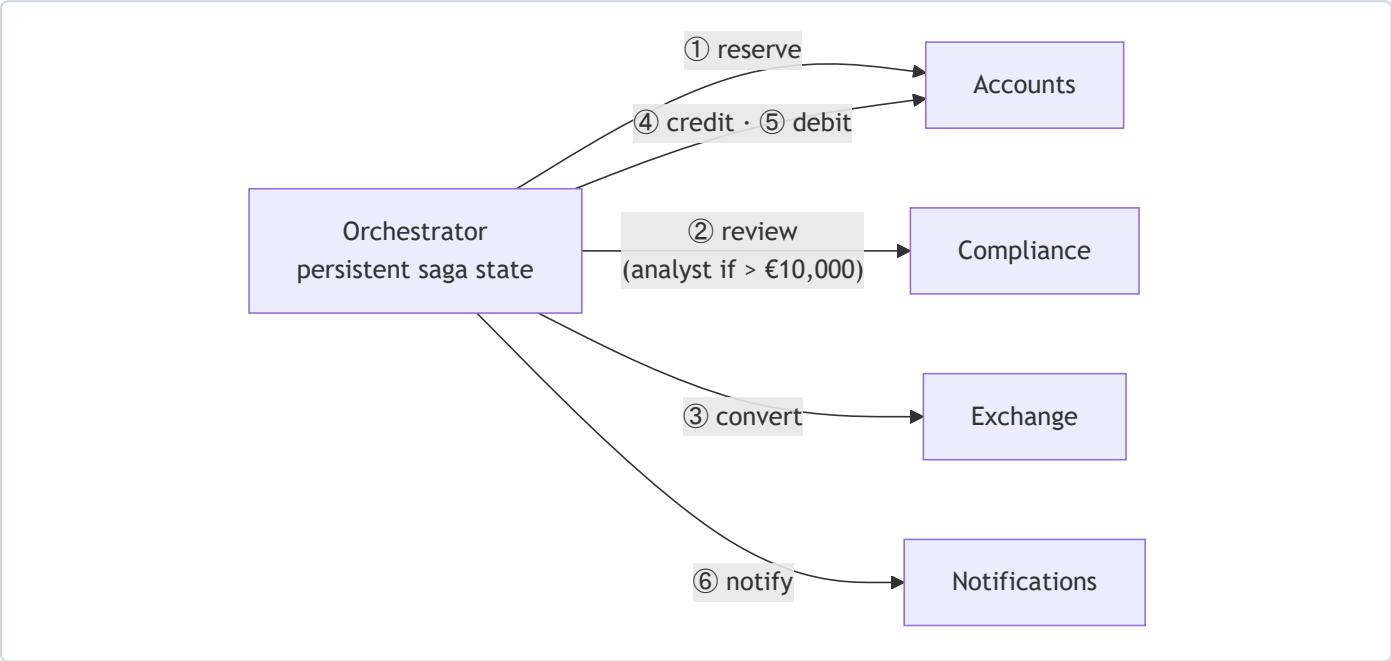
In .NET: *McMaster.NETCore.Plugins* wraps this mechanism.

Cheat sheet 14 · Orchestrated Saga

French name: *Saga orchestrée*

> **In one sentence**: an **orchestrator** drives a process spread over several services, remembers where it stands, retries, waits, and **compensates** in reverse order when needed.

Family: system decomposition · **Origin**: sagas, Garcia-Molina and Salem, 1987 · **Demo**: `14-SagaOrchestree` (port 5113) · 📖 Full page



Key points

- Several services = no single transaction: local steps + **compensations**.
- The process is **written in one place**; the state and history of each saga can be inspected.
- The engine handles **retries, (human) waits, deadlines, compensation** and recovery after a crash.
- Rule of thumb: choreography to **notify**, orchestration to **drive**.

✅ When to use it	❌ Watch out for
Long multi-step processes: order, international transfer, subscription	The orchestrator knows every participant (coupling)
Human waits, timeouts, precise failure rules	One more component to make highly available

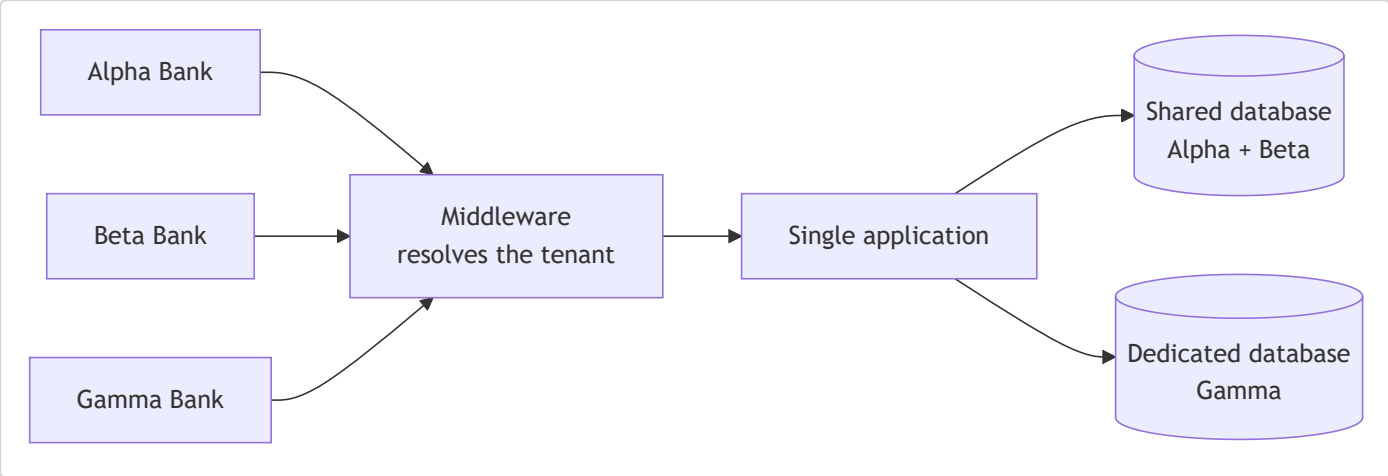
Pitfalls: participants **must be idempotent**; the "bloated conductor" that absorbs the participants' business logic.

In production: don't write your own engine → Temporal, Durable Functions, Step Functions, MassTransit / NServiceBus sagas. Compare with Choreography (same transfer).

Cheat sheet 15 · Multi-tenant (SaaS)

> **In one sentence**: **one** application, **one** deployment, for **several customers** (tenants), each with its own data, rules and quotas, never seeing anyone else's.

Family: system decomposition · **Demo**: `15-MultiTenant` (port 5114; tenant from the subdomain `beta.localhost:5114` or the `X-Locataire` header) · 📖 Full page



Key points

- The tenant is resolved **once** (middleware) and then **ambient** (`ContexteLocataire`).
- Safeguards: EF Core **global filter**, automatic **stamping** on write, **composite key** (tenant + id).
- Plans: shared database by default, **dedicated database** for those who pay for it; quotas against the **noisy neighbor**.
- `Mt.Locataires` contains no banking logic: the business code is written as if for a single bank.

✅ When to use it	❌ Cost
The same software sold to several organizations (B2B SaaS)	One isolation bug exposes **every** customer
A new customer in seconds, a single version	Migrations and backups across N dedicated databases

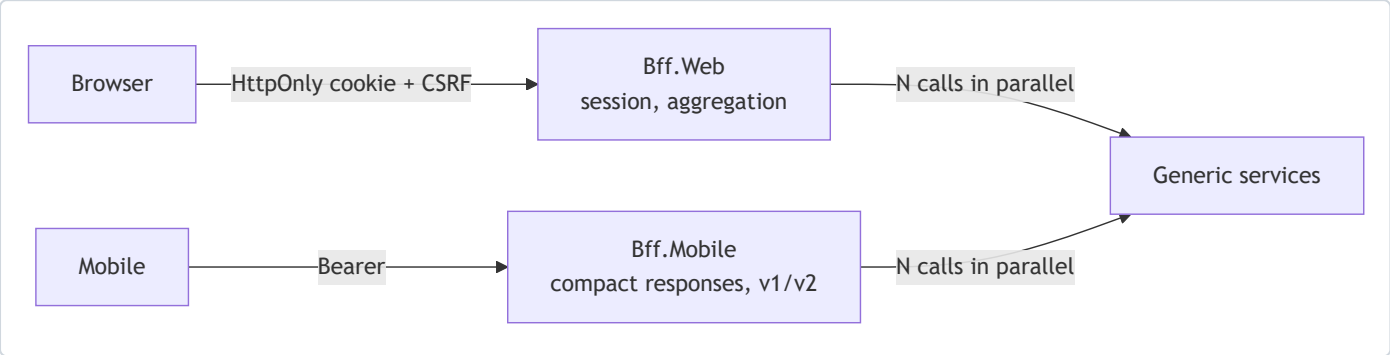
Pitfall: every request, background job, cache and log must **carry the tenant**.

In production: tenant taken from authentication, Finbuckle.MultiTenant, *Row-Level Security* as a second barrier.

Cheat sheet 16 · Backend for Frontend (BFF)

> **In one sentence**: **one backend per client application**, written by its team, that assembles the generic services into responses tailored to its screens and handles security its own way.

Family: system decomposition · **Origin**: SoundCloud, popularized by Sam Newman · **Demo**: `16-BFF` (web BFF 5115, mobile BFF 5116, services 5210; `alice / demo`) · 📖 Full page



Key points

- Without a BFF: the client chains calls (slow on mobile) or the services add per-screen routes.
- One call per screen; the BFF calls the services **in parallel** and degrades gracefully if one is down.
- On the web, the **token stays on the server** (session + HttpOnly cookie).
- BFFs don't reference each other: contracts are **copied**, only the useful fields (tolerant reader).

✅ When to use it	❌ Pointless if
Several client applications with different needs	A single application, a single backend (it **is** its BFF)
Single-page application that must not handle tokens	

Pitfalls: sneaking **business rules** into it (they stay in the services, which also check permissions); sessions to share between instances; intended duplication, to be accepted.

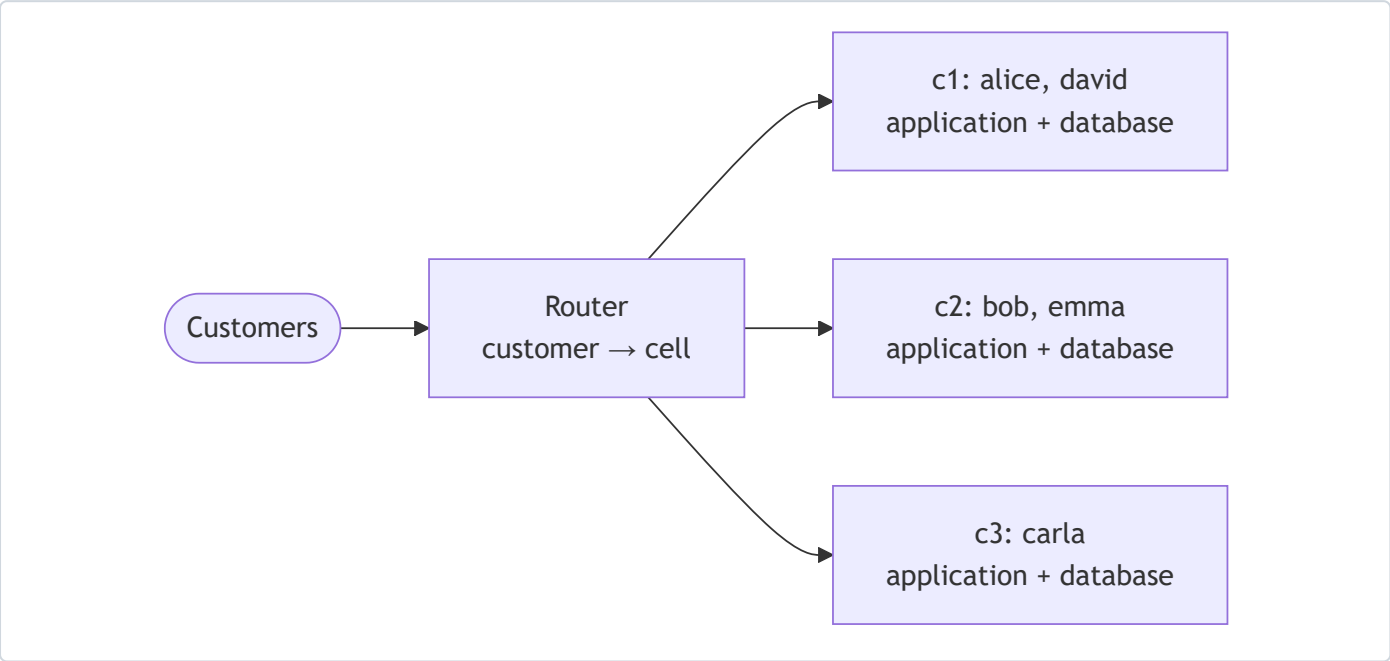
In production: Duende BFF, YARP, OpenID Connect. Alternative: GraphQL in front of the applications.

Cheat sheet 17 · Cell-Based Architecture

French name: *Architecture cellulaire*

> **In one sentence**: several **complete, independent copies** of the system, each serving part of the customers; a failure stays **locked inside its cell**.

Family: system decomposition · **Demo**: `17-CellBased` (router 5117, cells 5231–5234; start everything with `Cell.Lanceur`) · 📖 Full page (French)



Key points

- Goal: **bound the blast radius** (failure, bad deployment, greedy customer). 50 cells → an incident hits 2% of customers.
- Each cell contains **everything** and doesn't depend on the others.
- The router only **routes** (partition key → cell); a control plane places and moves customers.
- Deployment **in waves**: the first cell acts as the canary.

	Splits…	**A failure hits…**
Microservices	by function	one function, for everyone
Cells	by group of customers	everything, for one group

✅ **When to use it**	❌ **Overkill if**
Very large scale: a global outage costs more than N copies	Most systems: availability zones + progressive rollouts are enough
Natural partition key (the customer)	Lots of exchanges between customers

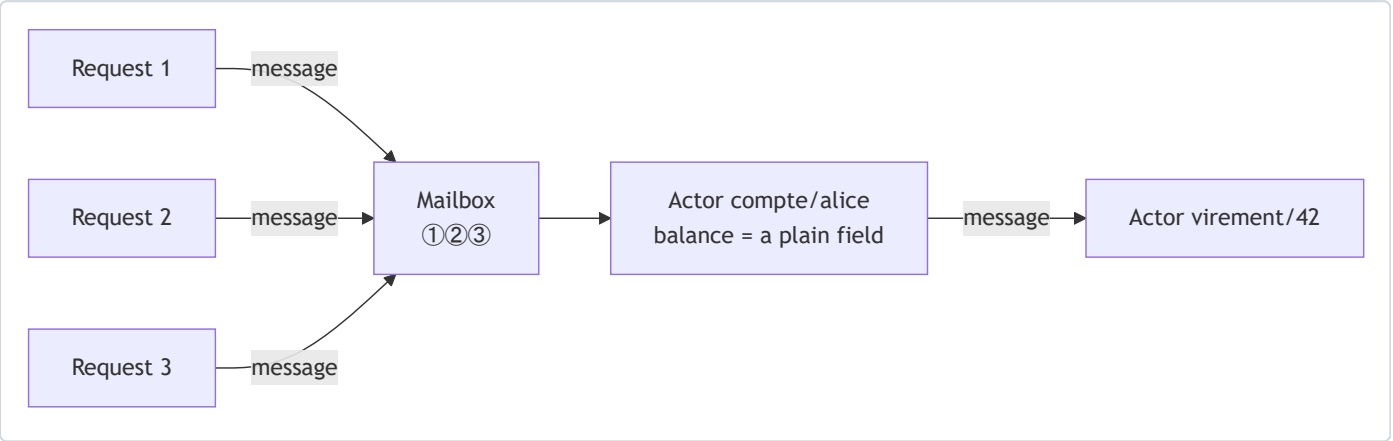
Pitfalls: **cross-cell** operations are slow and need compensation; a partition key commits you for a long time; customer migrations; a router that must be highly available.

Cheat sheet 18 · Actor Model

French name: *Modèle d'acteurs*

> **In one sentence**: thousands of actors with **private state**, communicating only **through messages** handled **one at a time**; no locks, and an actor that crashes is **replaced**.

Family: system decomposition · **Origin**: Carl Hewitt, 1973; popularized by Erlang · **Demo**: `18-ModeleActeurs` (port 5118; `/systeme` to observe and trigger failures) · 📖 Full page (French)



Key points

- **Every piece of data has a single owner**; safety is structural, not a matter of discipline.
- An actor can: change **its own** state, **send** messages, **create** actors (here *virtual* ones: you address them, the runtime creates them).
- **Different** actors work in parallel on all cores.
- On a failure, the **supervisor** replaces the actor, which reloads its durable state.

✅ When to use it	❌ Less suited if
Many independent entities whose state is often modified concurrently: games, IoT, sessions, accounts	Cross-cutting queries over relational data
Scaling across several machines	Transactions between entities are needed

Pitfalls: no transactions between actors (steps, idempotency, compensation); **request cycles = deadlocks**; a heavily solicited actor becomes a bottleneck; cross-cutting reads → a separate view.

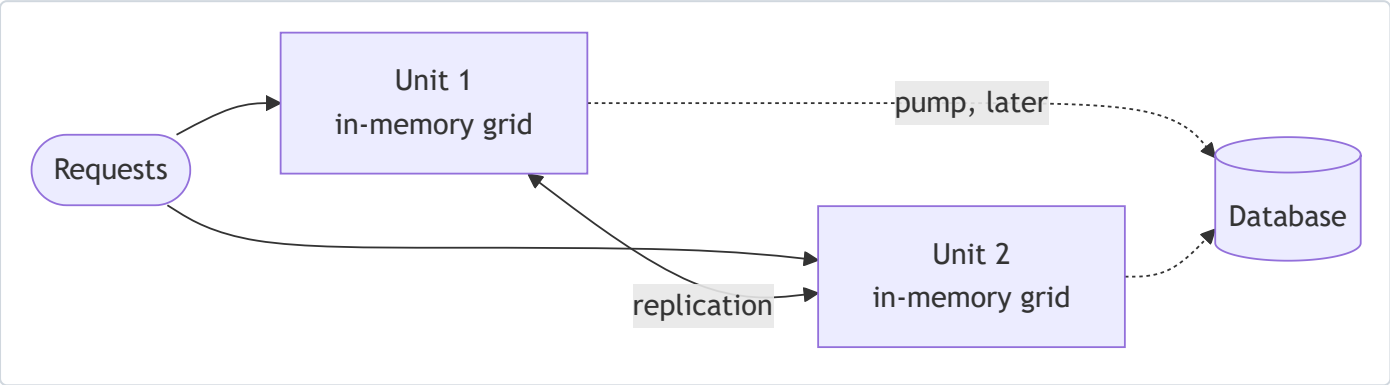
In .NET: Orleans (virtual actors), Akka.NET or Proto.Actor (fine-grained supervision).

Cheat sheet 19 · Space-Based Architecture

French name: *Architecture spatiale*

> **In one sentence**: the database leaves the request path; each **processing unit** keeps **all the data in memory**, replicated between units, and the database is updated in the background.

Family: system decomposition · **Origin**: *tuple spaces*; described notably by Mark Richards · **Demo**: `19-SpaceBased` (port 5119; `/grille` to observe and trigger failures) · 📖 Full page (French)



Key points

- A request only touches **memory**; the database is now just a **deferred backup**.
- **Elasticity**: a unit starts by copying another unit's grid; units are added and removed with the load.
- Safeguards: **versions** on every item and **affinity routing** (same key → same unit) to limit collisions.
- The database can go down without interrupting the service.

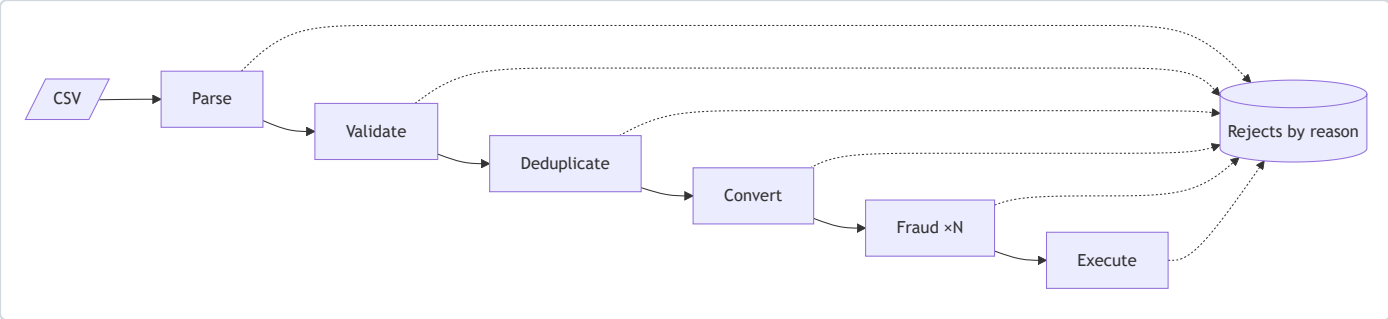
✅ When to use it	❌ Avoid if
Huge, unpredictable peaks: ticketing, auctions, bookings, games	Large data volumes (everything must fit in memory)
Data written mostly by key	A database that must be up to date at all times (accounting, real-time reports)

Pitfalls: **collisions**: accepted writes can be lost; the database lags behind for other systems; complex to test and operate. A poor choice for real balances: the demo uses them to **show** the collisions.

Cheat sheet 20 · Pipeline (pipes & filters)

> **In one sentence**: independent **filters** connected by **bounded pipes**, all working **at the same time** on a stream of data, like an assembly line.

Family: code organization · **Examples**: Unix shell, compilers, ETL · **Demo**: `20-Pipeline` (port 5120; `POST /imports` of a CSV file of transfers) · 📖 Full page (French)



Key points

- A filter knows neither its source nor its destination: it receives, produces or **rejects** (with a reason).
- A pipe is **bounded**: when full, it makes the previous stage wait (**back-pressure**) → bounded memory, whatever the file size.
- All stages work in parallel; to go faster, find the **bottleneck** and parallelize **that** stage only.
- Types checked at assembly; metrics per stage.

✅ When to use it	❌ Not for
Imports / exports, ETL, batches (payroll, invoicing), event streams, media	A whole interactive application
Any sequence of independent steps over data	Steps that need the whole data set

Pitfalls: throughput is that of the slowest stage; **stateful** filters (deduplication, execution) stay single and keep the order; a failure halfway leaves partial work → **idempotent** recovery.

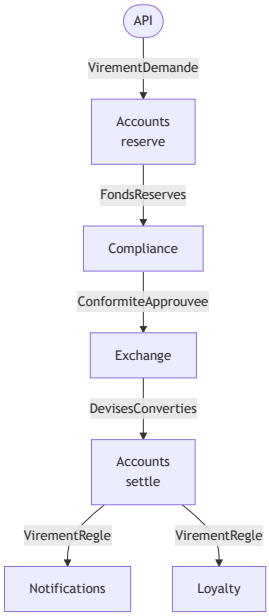
Takeaway: a pipeline lives **inside** a service or a module, itself organized according to another cheat sheet.

Cheat sheet 21 · Event-Driven Choreography

French name: *Chorégraphie (événementiel pur)*

> **In one sentence**: no service drives the process; each one **reacts to the events** of the others and publishes its own.

Family: system decomposition · **Demo**: `21-Choregraphie` (port 5121; `/bus` and `/chaos` to observe and break things) · 📖 Full page (French)



Key points

- The same transfer as the orchestrated saga, but the process is **written nowhere**: it emerges from the subscriptions.
- A service only knows **event contracts** (architecture test: no service references another).
- Adding a reaction = **subscribing**, without touching the others.
- "Where is transfer 42?" → a **projection** to build (the Tracking).

✅ When to use it	❌ Prefer orchestration if
Reactions to a fact: notify, bill, archive, synchronize	Many steps, waits, timeouts, failure rules
Short, stable processes with few participants	The process must be easy to read and change

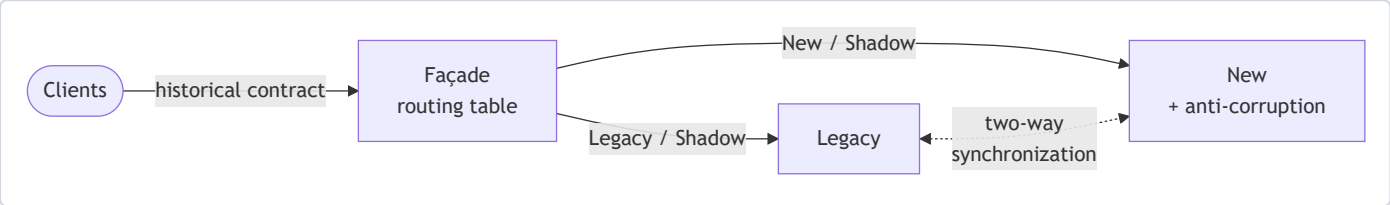
Pitfalls: compensations and timeouts are **scattered**; a failure shows up as **silence** (monitor dead letters and stuck transfers); single point of failure: the broker.

Common combination: an orchestrator drives, and publishes events that others choreograph.

Cheat sheet 22 · Strangler Fig (progressive migration)

> **In one sentence**: a **façade** in front of the legacy system takes its features away **one route at a time** in favor of the new one, until the legacy can be switched off.

Family: migration · **Origin**: Martin Fowler, 2004 · **Demo**: `22-StranglerFig` (façade 5122, legacy 5240, new 5241) · 📖 Full page (French)



Key points

- Each route has a mode: **Legacy**, **Shadow** (the legacy answers, the new one is called and compared), **New**.
- The routing table can be changed **live**: every step can be undone **with one request**; anything undeclared stays on the legacy.
- The new system has its own model and database, behind an **anti-corruption layer**, and reproduces the historical contract exactly.
- No project references the legacy: when the day comes, you delete its project and the anti-corruption layer.

✅ When to use it	❌ Less useful if
A production system that can be neither stopped nor frozen	Small system rewritten in a few weeks
Features that can be split into routes, screens or messages	No entry point to intercept

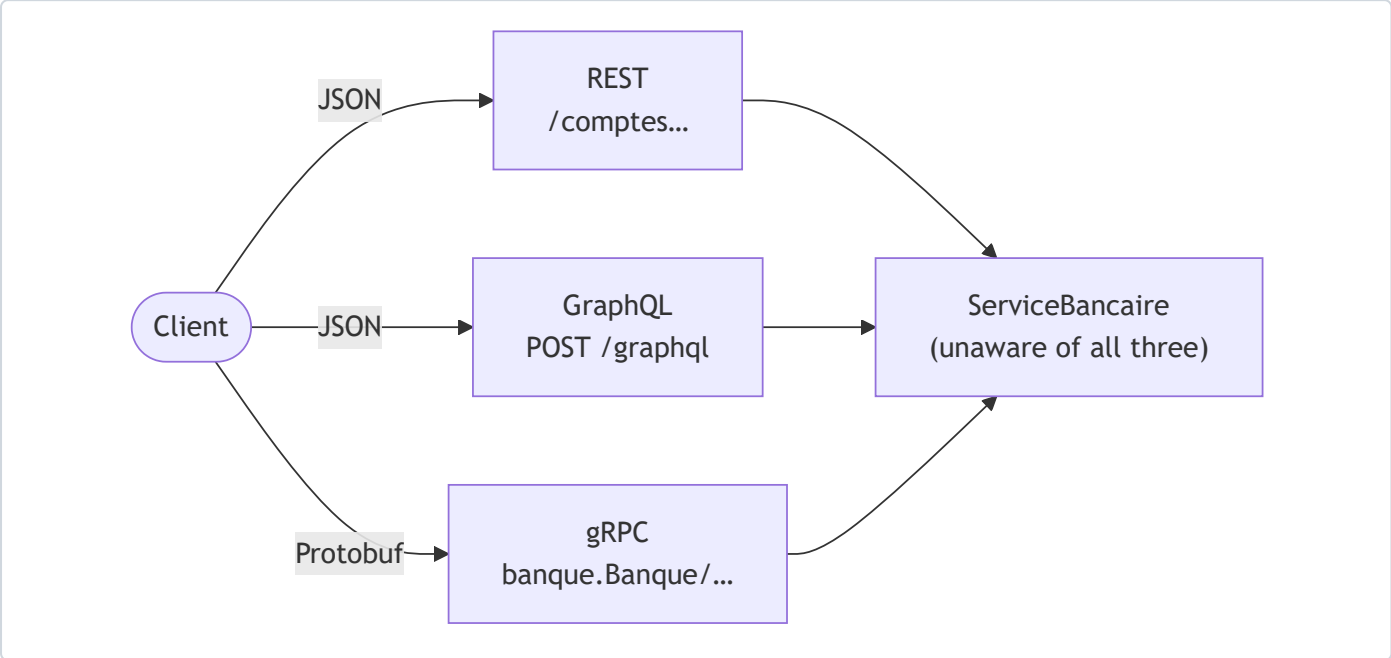
Pitfalls: synchronization (loops, failures, lag); switch data over **in consistent blocks**; the legacy contract must be reproduced, quirks included; a migration stopped at 80% = the worst of both worlds.

In production: YARP, Debezium, Scientist / Diffy for shadowing, feature flags.

Cheat sheet 23 · REST, gRPC, GraphQL

> **In one sentence**: the same business logic exposed as **resources** (REST), as **procedures** (gRPC) and as a **graph the client queries** (GraphQL).

Family: exposure · **Demo**: `23-RestGrpcGraphQL` (REST and GraphQL 5123 over HTTP/1.1, gRPC 5250 over HTTP/2; Protobuf, gRPC and GraphQL hand-written) · 📖 Full page (French)



Key points

	REST	**gRPC**	**GraphQL**
You manipulate	resources + HTTP verbs	procedures	a graph of types
Contract	implicit or OpenAPI	`.proto`, generated code	SDL schema, validated before execution
Response shape	chosen by the server	chosen by the server	**chosen by the client**
Format / transport	JSON, any HTTP	binary, **HTTP/2** required	JSON, `POST`
Errors	HTTP status + `problem+json`	gRPC status in the trailers	`200 + errors`
HTTP cache · streaming · browser	✅ · ❌ · ✅	❌ · ✅ · ❌ (gRPC-Web)	❌ · subscriptions · ✅

When to choose what

- **REST**: public or partner API, simple resources, caching.
- **gRPC**: between internal services; throughput, latency, streaming, strict contracts between teams.
- **GraphQL**: many clients with rich screens; one round trip per screen, instead of one BFF per application.

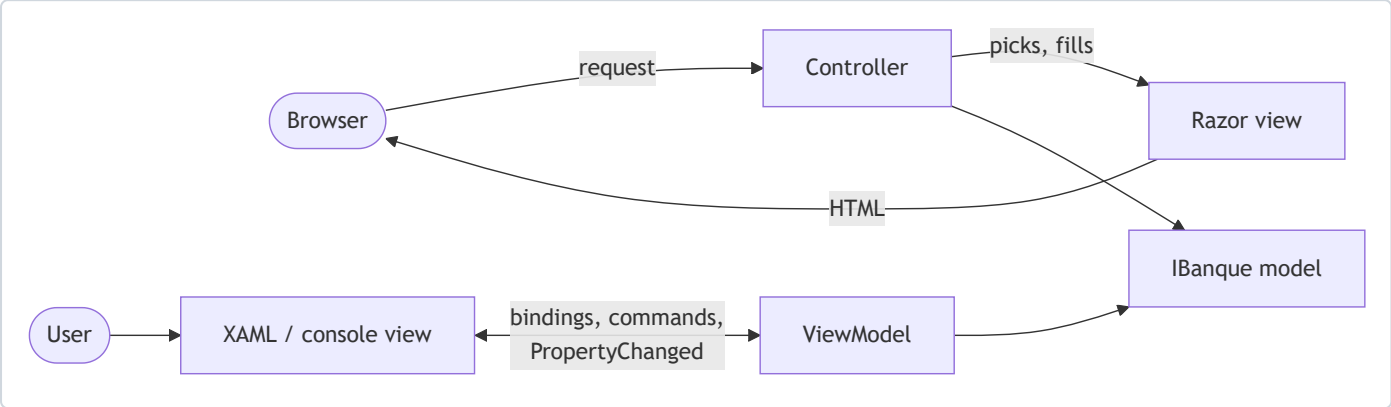
Pitfalls: REST → round trips and over-fetching; gRPC → unreadable without tooling, no browser; GraphQL → **hidden N+1** (DataLoader), queries to **bound** (depth, cost), HTTP cache lost.

Takeaway: the style is chosen **boundary by boundary**; the three often coexist.

Cheat sheet 24 · MVC and MVVM

> **In one sentence**: the same screen in web **MVC** (a controller receives the request and picks a view, discarded after rendering) and in **MVVM** (a living ViewModel that the view observes through bindings).

Family: presentation · **Demo**: `24-MvcMvvm` (MVC site 5124; WPF views, Windows only, and a console on the same ViewModels) · 📖 Full page (French)



Key points

	MVC	MVVM
Receives the action	the controller	the view, which passes it on via a binding or a command
Who knows whom	the controller picks the view	the view knows the ViewModel, **not the other way around**
Screen lifetime	**one request**	**as long as the window is open**
Validation	after submission (ModelState)	**on every keystroke**
Testable without a screen	the controller's decision	**all of the screen's behavior**

- The model knows nothing about screens (architecture test); the ViewModels know nothing about WPF (another test).
- XAML bindings are not compiled: a test **reads the XAML** to check them.
- Vocabulary trap: ASP.NET MVC "view models" are plain data bags, not MVVM ViewModels.

✅ MVC	✅ MVVM
Server-rendered site, SEO, holding up under load	Desktop (WPF, WinUI, Avalonia), mobile (MAUI)

Pitfalls: MVC → one page per step, state in hidden fields. MVVM → leaking subscriptions, **stale asynchronous responses**, the UI thread, ceremony (CommunityToolkit.Mvvm helps).

Takeaway: MVC and MVVM are only the **presentation layer**; the model behind them follows any other cheat sheet.